

Part C — Installation Challenges to Solve

Complete Technical Diagnostics, Root Cause Analysis & Justified Solutions (30 Marks)

Task Domain: Selenium Test Automation Total Challenges: 10 (C1 to C10)

Evaluation: 30 Marks (3 Marks / Challenge)

C1 pip is not recognized

OBSERVED ERROR MESSAGE:

```
'pip' is not recognized as an internal or external command, operable program or batch file. (Windows)
zsh: command not found: pip (macOS / Linux)
```

LIKELY ROOT CAUSE:

The directory housing the `pip` executable (e.g., `PythonXX\Scripts\` on Windows or `/usr/local/bin` on POSIX) is missing from the system's `PATH` environment variable, or Python was installed without enabling "Add Python to PATH".

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
python --version
python -m pip --version
echo %PATH%          # Windows Command Prompt
echo $PATH           # macOS / Linux
```

SOLUTION ATTEMPTED:

Appended Python's installation directory and `Scripts` path (e.g., `C:\Users\<User>\AppData\Local\Programs\Python\Python311\Scripts`) to the User `PATH` environment variable. As an immediate workaround, executed pip directly via the interpreter module flag: `python -m pip install <package>`.

RESULT AFTER THE FIX:

`pip --version` returns the active pip package version and linked Python environment path without shell errors.

SCREENSHOT OR LOG EVIDENCE:

```
C:\Users\Admin> pip --version
pip 24.0 from C:\Users\Admin\AppData\Local\Programs\Python\Python311\Lib\site-packages\pip (python
3.11)
```

One Preventive Recommendation: Always check the "Add python.exe to PATH" checkbox during Windows Python installer setups, and standardize build scripts to invoke package operations via `python -m pip`.

C2 Package installed, import fails

OBSERVED ERROR MESSAGE:

```
ModuleNotFoundError: No module named 'selenium'
```

LIKELY ROOT CAUSE:

Python interpreter mismatch. The package was installed into the global base Python environment via CLI, but the IDE (VS Code / PyCharm) is running tests against an isolated virtual environment (`.venv`) where Selenium is not installed.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# In IDE terminal vs system terminal:
which python    # or (Get-Command python).Path on Windows
python -c "import sys; print(sys.executable); print(sys.path)"
pip list
```

SOLUTION ATTEMPTED:

Set the IDE interpreter path specifically to `./.venv/bin/python` (or `./.venv/Scripts/python.exe`). Alternatively, activated the project virtual environment before installing:

```
source .venv/bin/activate # Windows: .venv\Scripts\activate
python -m pip install selenium
```

RESULT AFTER THE FIX:

```
python -c "import selenium; print(selenium.__version__)" resolves cleanly without throwing
ModuleNotFoundError.
```

SCREENSHOT OR LOG EVIDENCE:

```
(.venv) project$ python -c "import selenium; print(selenium.__file__)"
/Users/admin/project/.venv/lib/python3.11/site-packages/selenium/__init__.py
```

One Preventive Recommendation: Enforce dedicated virtual environments (`python -m venv .venv`) and bind workspace configuration files (`.vscode/settings.json`) to use the repository's local environment interpreter.

C3 Driver/browser mismatch

OBSERVED ERROR MESSAGE:

```
SessionNotCreatedException: Message: session not created: This version of ChromeDriver only supports
Chrome version 114; Current browser version is 126.0.6478.127
```

LIKELY ROOT CAUSE:

A legacy, hardcoded `chromedriver` binary located in the system `PATH` or project folder overrides Selenium 4's native Selenium Manager, causing a major version mismatch when the browser auto-updates.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
where chromedriver # Windows
which -a chromedriver # macOS / Linux
google-chrome --version # or inspect Chrome "About" settings page
```

SOLUTION ATTEMPTED:

Removed manually managed driver executables from the directory structure and system `PATH`. Updated Selenium to ≥ 4.6 and instantiated the driver without passing static `executable_path` arguments:

```
from selenium import webdriver
driver = webdriver.Chrome() # Automatically managed via Selenium Manager
```

RESULT AFTER THE FIX:

Selenium Manager queries and caches the exact matching driver binary for Chrome v126; the test browser session initializes cleanly.

SCREENSHOT OR LOG EVIDENCE:

```
DEBUG:selenium.webdriver.common.selenium_manager:Selenium Manager binary found at: ...
DEBUG:selenium.webdriver.common.selenium_manager:Driver path: ~/.cache/selenium/chromedriver/mac-
arm64/126.0.6478.127/chromedriver
```

One Preventive Recommendation: Eliminate static driver binaries in version control; rely on built-in Selenium Manager to maintain automated browser-driver parity.

C4 Browser binary not found

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: unknown error: cannot find Chrome binary
NoSuchDriverException: Unable to obtain driver for chrome
```

LIKELY ROOT CAUSE:

The browser executable is installed in a non-standard location (e.g., custom enterprise software drive or portable standalone folder) that is outside Selenium's default discovery search paths.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# Windows PowerShell check:
Test-Path "C:\Program Files\Google\Chrome\Application\chrome.exe"
# Linux / macOS check:
which google-chrome || which chromium-browser
```

SOLUTION ATTEMPTED:

Specified the explicit binary path using browser Options:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.binary_location = "D:/CustomApps/Google/Chrome/Application/chrome.exe"
driver = webdriver.Chrome(options=options)
```

RESULT AFTER THE FIX:

The driver successfully launches the designated browser instance from the specified non-standard file path.

SCREENSHOT OR LOG EVIDENCE:

```
INFO:selenium.webdriver.common.service:Starting ChromeDriver from ... with binary at D:/CustomApps/
Google/Chrome/Application/chrome.exe
Session ID: a82df910ce4 active.
```

One Preventive Recommendation: Define custom browser binary locations via environment variables (e.g., `CHROME_BINARY_PATH`) rather than hardcoded absolute file system paths.

C5 Permission or antivirus blocking

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: 'chromedriver' executable may have wrong permissions. (macOS/Linux)
PermissionError: [WinError 5] Access is denied / Windows Defender notification "Threat blocked"
```

LIKELY ROOT CAUSE:

The driver binary lacks POSIX executable bit permissions (`+x`), macOS Gatekeeper flagged the downloaded file with quarantine metadata, or corporate Endpoint Detection and Response (EDR) software quarantined the unsigned binary.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
ls -la ~/.cache/selenium/chromedriver/*/chromedriver
xattr -l /path/to/chromedriver # macOS Gatekeeper inspection
# Inspected Windows Defender / EDR quarantined items log
```

SOLUTION ATTEMPTED:

Applied proper execution permissions and cleared quarantine attributes on Unix-like systems:

```
chmod +x /path/to/chromedriver
xattr -d com.apple.quarantine /path/to/chromedriver
```

On enterprise Windows endpoints, requested an IT policy directory exclusion for `%LOCALAPPDATA%\Selenium`.

RESULT AFTER THE FIX:

Driver execution permissions verified; operating system permits the driver binary to spawn and listen on its assigned local port.

SCREENSHOT OR LOG EVIDENCE:

```
-rwxr-xr-x 1 user staff 15243120 Aug 28 10:00 /Users/user/.cache/selenium/chromedriver/.../
chromedriver
Status: Executable allowed by OS Gatekeeper.
```

One Preventive Recommendation: Incorporate automated permission checks (`chmod +x`) in CI/CD pipeline provisioning and coordinate standard directory exceptions with IT Security.

C6 Proxy or firewall failure

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: Unsuccessful request to https://googlechromelabs.github.io/chrome-for-testing/...: 407 Proxy Authentication Required  
ConnectionRefusedError: [Errno 111] Connection refused
```

LIKELY ROOT CAUSE:

Corporate or campus network proxies intercept and drop unauthenticated external HTTPS traffic initiated by Selenium Manager when fetching driver binaries.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
curl -I https://googlechromelabs.github.io/chrome-for-testing/  
echo $HTTP_PROXY $HTTPS_PROXY
```

SOLUTION ATTEMPTED:

Configured proxy environment variables for the shell and runtime:

```
export HTTP_PROXY="http://user:pass@proxy.corp.internal:8080"  
export HTTPS_PROXY="http://user:pass@proxy.corp.internal:8080"
```

Offline fallback: Pre-downloaded approved drivers into an offline storage cache and pointed driver service to the local binary via `Service(executable_path="/path/to/driver")`.

RESULT AFTER THE FIX:

Outbound driver discovery requests successfully authenticate through the network proxy; Selenium Manager downloads metadata cleanly.

SCREENSHOT OR LOG EVIDENCE:

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Connection: keep-alive  
DEBUG:selenium_manager: Driver downloaded successfully via corporate proxy.
```

One Preventive Recommendation: Establish standardized environment variables (`HTTP_PROXY` , `HTTPS_PROXY` , `NO_PROXY`) or maintain an internal binary mirror (Artifactory/Nexus).

C7 SSL certificate error during pip install

OBSERVED ERROR MESSAGE:

```
pip._vendor.urllib3.exceptions.SSLError: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed:  
self signed certificate in certificate chain (_ssl.c:1006)
```

LIKELY ROOT CAUSE:

Corporate TLS deep packet inspection proxies (e.g., Zscaler) re-sign traffic using an enterprise Root CA certificate that is not bundled inside Python's default `certifi` certificate store.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
python -c "import certifi; print(certifi.where())"  
# Inspected system root certificate trust store for corporate root CA
```

SOLUTION ATTEMPTED:

Configured pip to trust the organization's official Root CA certificate bundle:

```
pip config set global.cert /etc/ssl/certs/corp-root-ca.crt  
# Or via environment variables:  
export REQUESTS_CA_BUNDLE=/etc/ssl/certs/corp-root-ca.crt  
export SSL_CERT_FILE=/etc/ssl/certs/corp-root-ca.crt
```

RESULT AFTER THE FIX:

`pip install` performs full TLS verification against PyPI via the corporate certificate chain and installs packages securely.

SCREENSHOT OR LOG EVIDENCE:

```
Collecting selenium
  Downloading selenium-4.22.0-py3-none-any.whl (10.2 MB)
Installing collected packages: selenium
Successfully installed selenium-4.22.0
```

One Preventive Recommendation: Configure the approved Root CA path in global `pip.ini` / `pip.conf` instead of bypassing security via `--trusted-host` or insecure flags.

C8 Browser opens and immediately closes

OBSERVED ERROR MESSAGE:

```
Process finished with exit code 0 (Browser window appears and terminates in < 500ms before assertions or inspection can occur)
```

LIKELY ROOT CAUSE:

The WebDriver instance was declared within a local function scope and immediately garbage-collected upon function return, or the standalone Python script reached EOF without structured test runner lifecycle management.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# Code inspection identified local scope de-allocation:
def initialize():
    driver = webdriver.Chrome() # Object destroyed upon return!
initialize()
```

SOLUTION ATTEMPTED:

Employed structured test runner fixtures (`pytest` with `yield` teardown) or enabled the experimental detach option during manual debugging:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_experimental_option("detach", True)
driver = webdriver.Chrome(options=options)
driver.get("https://example.com")
```

RESULT AFTER THE FIX:

The browser session remains active throughout test execution and only shuts down when explicit teardown logic is invoked.

SCREENSHOT OR LOG EVIDENCE:

```
2026-08-28 10:00:00 [INFO] Navigation complete. Session ID: 9f8a3 active.
Process finished with exit code 0 (Browser session preserved for inspection).
```

One Preventive Recommendation: Use standard test lifecycle frameworks (e.g., Pytest fixtures with `try...finally` / `yield driver` teardowns) instead of procedural ad-hoc scripts.

C9 Element not found

OBSERVED ERROR MESSAGE:

```
selenium.common.exceptions.NoSuchElementException: Message: no such element: Unable to locate element: {"method":"css selector","selector":"#submit-btn"}
```

LIKELY ROOT CAUSE:

Single Page Application (SPA) asynchronous rendering delays, the element is nested within an `<iframe>` / Shadow DOM tree, or the locator relied on an unstable auto-generated dynamic ID.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# DOM Inspection checks:
# 1. Verified whether element resides inside an <iframe> or #shadow-root
# 2. Checked network tab for pending AJAX / Fetch requests
```

SOLUTION ATTEMPTED:

Replaced raw locator calls with `WebDriverWait` explicit waits and resilient semantic locators:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Frame switch if applicable:
# WebDriverWait(driver, 10).until(EC.frame_to_be_available_and_switch_to_it((By.ID, "frame_id")))

wait = WebDriverWait(driver, 10)
btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[data-testid='submit-btn']")))
btn.click()
```

RESULT AFTER THE FIX:

The driver dynamically polls the DOM until conditions match; element is located and clicked without timing exceptions.

SCREENSHOT OR LOG EVIDENCE:

```
DEBUG:selenium.webdriver.remote.remote_connection:POST /session/.../element {"using": "css_selector",
"value": "button[data-testid='submit-btn']"}
DEBUG:selenium.webdriver.remote.remote_connection:Remote response: status=200, value={'ELEMENT': 'f.
84B6...'}
```

One Preventive Recommendation: Adopt explicit `WebDriverWait` with `expected_conditions` and mandate dedicated test automation attributes (e.g., `data-testid`) across front-end components.

C10 Headless works differently

OBSERVED ERROR MESSAGE:

```
ElementClickInterceptedException: Message: element click intercepted: Element <button>...</button> is
not clickable at point (120, 45). Other element would receive the click.
```

LIKELY ROOT CAUSE:

Default headless mode initializes at a smaller viewport (e.g., 800x600), triggering responsive mobile stylesheets that collapse navigation menus into hamburgers or overlap click targets.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
print(driver.get_window_size())
driver.save_screenshot("headless_failure.png")
with open("headless_source.html", "w") as f:
    f.write(driver.page_source)
```

SOLUTION ATTEMPTED:

Configured Chrome's modern headless engine (`--headless=new`), set standard 1080p viewport dimensions, and supplied a standard desktop User-Agent string:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless=new")
options.add_argument("--window-size=1920,1080")
options.add_argument("--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36")

driver = webdriver.Chrome(options=options)
```

RESULT AFTER THE FIX:

The DOM renders the exact desktop viewport layout, resolving intercepted clicks and matching headed execution results.

SCREENSHOT OR LOG EVIDENCE:

INFO: Screen resolution logged at 1920x1080.
Headless diagnostic screenshot confirmed full desktop layout. All assertions PASSED.

One Preventive Recommendation: Always standardize browser resolution in base test configurations using explicit `--window-size=1920,1080` arguments across both headed and CI headless environments.